

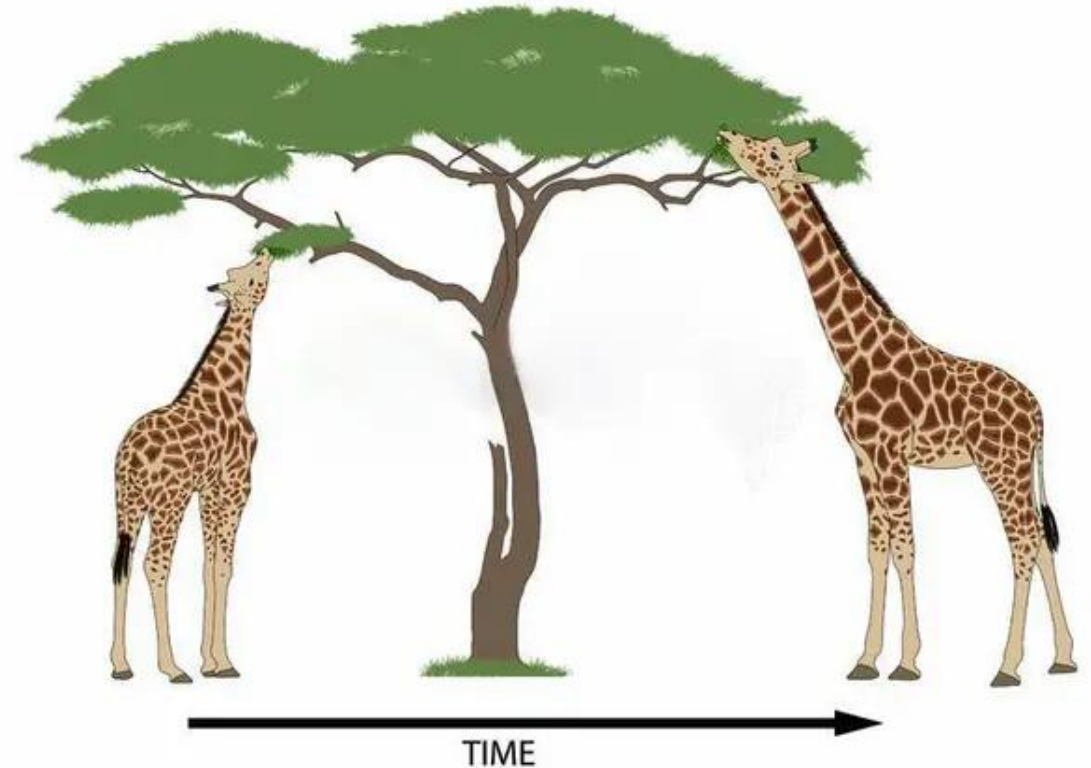
241-405 Optimization
Nature-Inspired Optimization 2
Genetic Algorithm
Yossawee Keaomanee

Outline

- Introduction
- Basic Steps of Genetic Algorithm
- Typical Applications
- Advantages & Disadvantages

Introduction

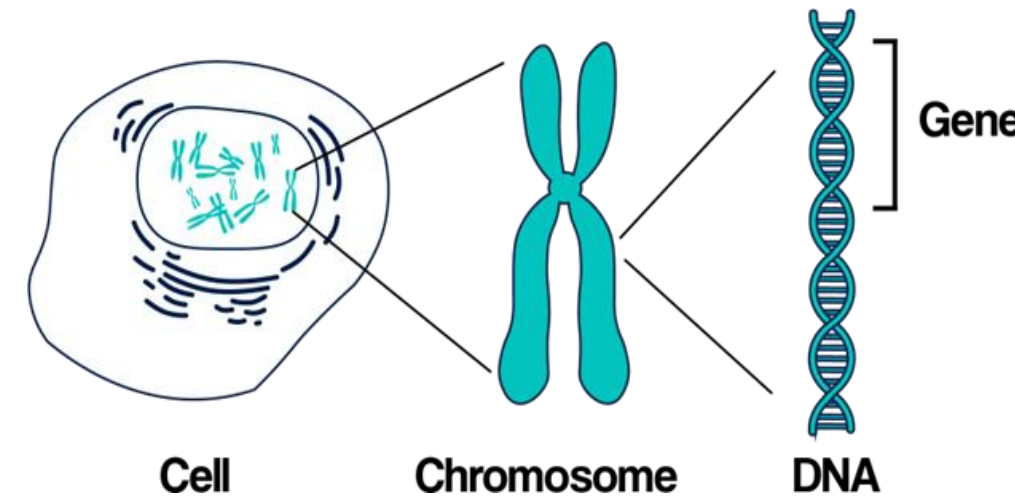
- ยีราฟอาศัยอยู่ในพื้นที่ที่มีต้นไม้สูง จึงจำเป็นต้องมีคอที่ยาวเพื่อเอื้อมไปถึงใบไม้
- บรรพบุรุษของยีราฟมีทั้งคอยาวและคอสั้น ซึ่งเกิดจากการกลายพันธุ์
- ยีราฟคอยาวได้เปรียบในการหาอาหาร การต่อสู้ และการผสมพันธุ์ ยีราฟคอสั้นต้องดิ้นรนเพื่อความอยู่รอดและการสืบพันธุ์
 - ยีราฟคอยาวกลายเป็นกลุ่มเด่น
 - ยีราฟคอสั้นสูญพันธุ์



<https://www.polygence.org/projects/research-project-natural-selection-in-animals>

Introduction

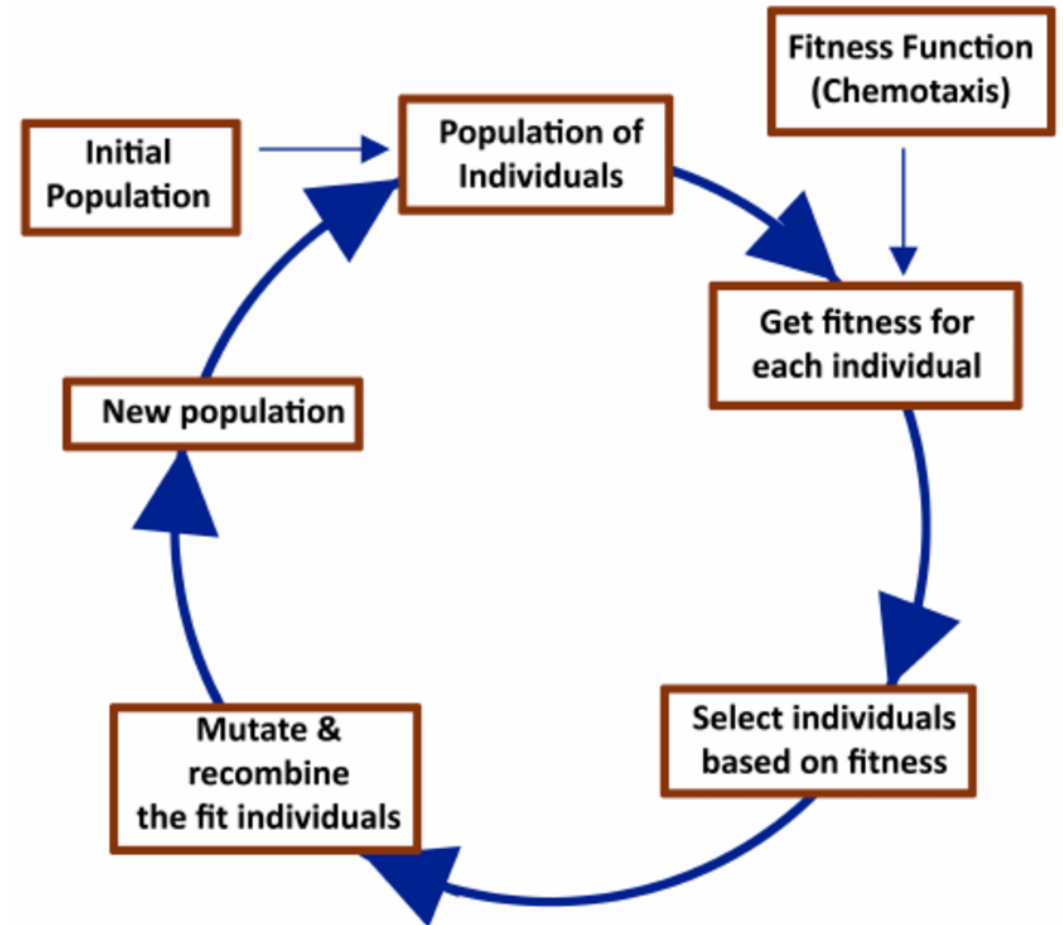
- ร่างกายของสัตว์ประกอบไปด้วยเซลล์ซึ่งมีนิวเคลียสที่บรรจุโครโมโซมและยีน (DNA) อยู่ภายใน
- ยีน ทำหน้าที่ควบคุมลักษณะทางกายภาพและพฤติกรรมของสิ่งมีชีวิต
- การรวมกลุ่มใหม่ของยีน (การครอสโอเวอร์ Cross over) ทำให้เกิดลูกหลานที่มีลักษณะผสมผสานจากพ่อและแม่
- การกลายพันธุ์ทำให้เกิดการเปลี่ยนแปลงแบบสุ่มในยีน ซึ่งสร้างความหลากหลายและอาจนำไปสู่การเกิดสปีชีส์ใหม่
- กระบวนการเหล่านี้ถือเป็นพื้นฐานของทฤษฎีการคัดเลือกโดยธรรมชาติของดาร์วิน



Introduction

อัลกอริทึมพันธุกรรม (Genetic Algorithm หรือ GA)

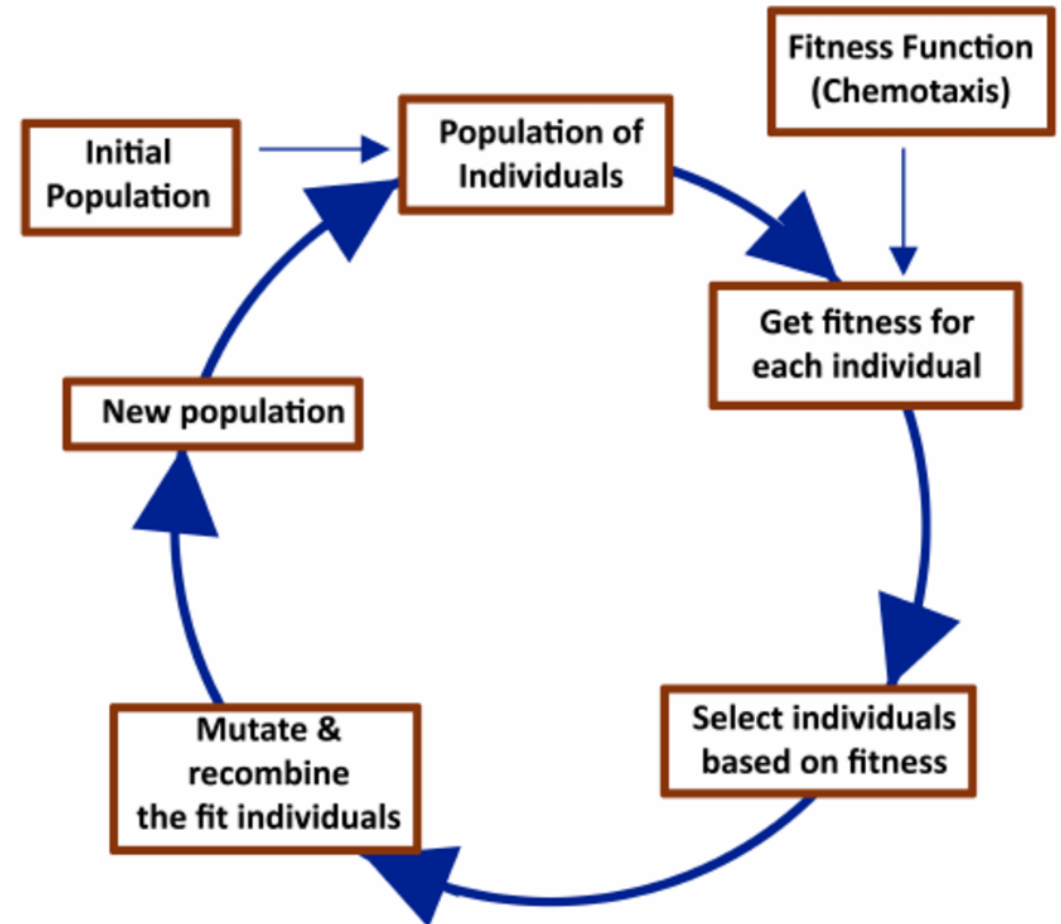
- ได้รับแรงบันดาลใจมาจากระบวนการคัดเลือกโดยธรรมชาติในทางชีววิทยา
- **ใช้เพื่อหาคำตอบโดยประมาณ** สำหรับปัญหาที่มีความซับซ้อน ซึ่งอาจยากต่อการแก้ไขด้วยวิธีการแบบดั้งเดิม.



Introduction

เป็นอัลกอริทึมเชิงวิวัฒนาการ
(Evolutionary Algorithms) ซึ่ง
เลียนแบบวิวัฒนาการทางชีวภาพผ่าน
กระบวนการต่าง ๆ เช่น

- การคัดเลือก (Selection)
- การผสมข้ามสายพันธุ์
(Crossover/Recombination)
- การกลายพันธุ์ (Mutation)



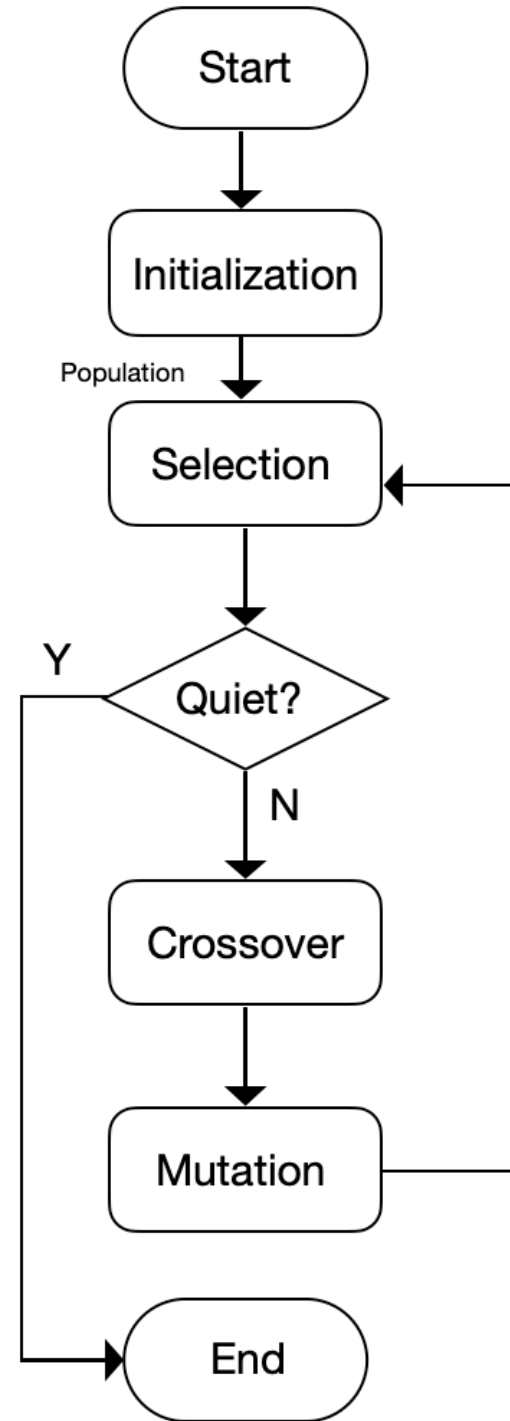
Basic Steps of Genetic Algorithm

ขั้นตอนที่ 1 : เริ่มต้น (Initialization) : สร้างประชากรเริ่มต้นของคำตอบที่เป็นไปได้ด้วยการสุ่ม

ขั้นตอนที่ 2 : ประเมินผล (Evaluation) : ประเมินคำตอบแต่ละชุดด้วยฟังก์ชันความเหมาะสม (Fitness Function) เพื่อตัดสินว่าคำตอบนั้นดี เพียงใด

ขั้นตอนที่ 3 : คัดเลือก (Selection) : คัดเลือกชุดคำตอบที่มีความเหมาะสม

ขั้นตอนที่ 4 : ผสมข้ามสายพันธุ์ (Crossover/Recombination) : รวมส่วนประกอบของคำตอบจากพ่อแม่สองชุดเข้าด้วยกันเพื่อสร้างชุดคำตอบใหม่ (ลูกหลาน)

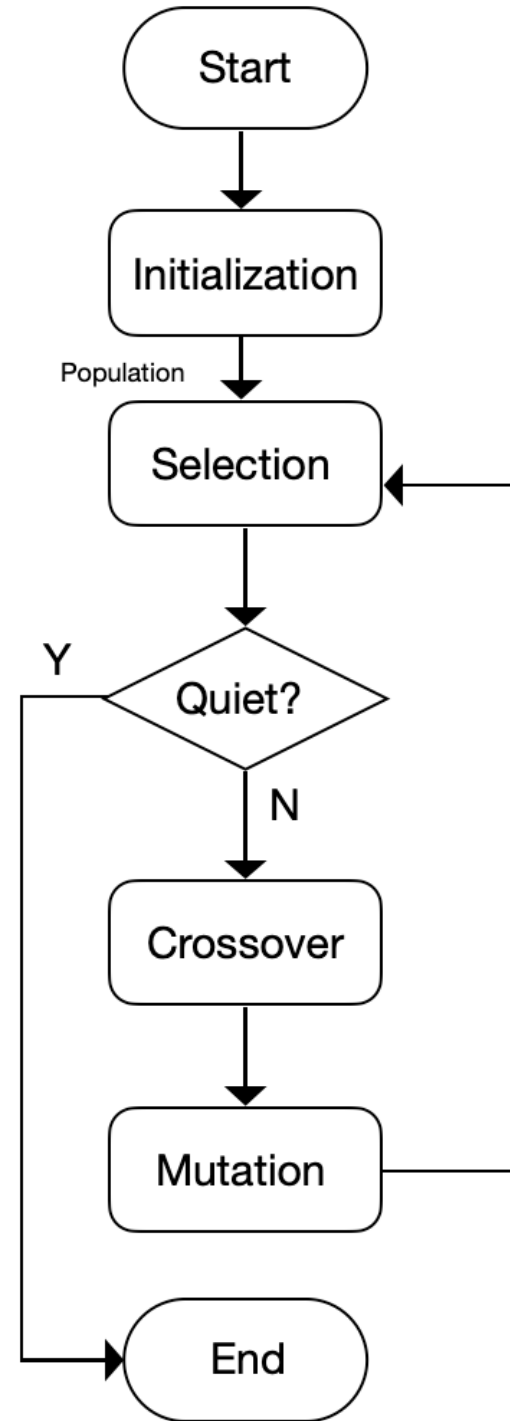


Basic Steps of Genetic Algorithm

ขั้นตอนที่ 5 : กลายพันธุ์ (Mutation) : สุ่มเปลี่ยนแปลงบางส่วนของคำตอบ เพื่อรักษาความหลากหลายทางพันธุกรรม

ขั้นตอนที่ 6 : แทนที่ (Replacement) : แทนที่สมาชิกในประชากรที่มีความเหมาะสมน้อยด้วยลูกหลานใหม่ที่เพิ่งสร้างขึ้น

ขั้นตอนที่ 7 : ทำซ้ำ (Repeat) : ทำซ้ำกระบวนการเดิมต่อไปหลายรุ่น จนกว่าจะบรรลุเงื่อนไขการหยุดที่กำหนดไว้ (เช่น ครบจำนวนรุ่นที่ตั้งไว้สูงสุด หรือได้ค่าความเหมาะสมที่น่าพอใจ)



Learn from Example

หาค่าของฟังก์ชันที่เหมาะสมที่สุด (Best fit) โดยใช้กระบวนการทางพันธุกรรม (Genetic Algorithm)

เป้าหมาย: คำนวณค่าจำนวนเต็มของ a, b, c, d ที่ทำให้:

$$2a + 4b + 3c - 2d = 41$$

- ใช้กระบวนการทางพันธุกรรม (Genetic Algorithm) เพื่อประมาณการหาคำตอบที่ถูกต้องที่สุด.

Step 1: Initialization (Initial Population)

สุ่มสร้างกลุ่มของคำตอบที่เป็นไปได้ (ประชากร) โดยแต่ละคำตอบคือ โครโมโซม หนึ่งแท่ง ซึ่งประกอบด้วย 4 ยีน (แทนค่าตัวแปร a , b , c , และ d).

- แต่ละยีนสามารถเป็นจำนวนเต็มในช่วงที่กำหนดไว้ล่วงหน้า เช่น $(-10, 10)$
- กำหนดประชากรเริ่มต้นจำนวน 6 โครโมโซม

<i>Chromosome ID</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>
1	4	3	2	0
2	-3	5	0	-1
3	2	3	-1	-2
4	-3	5	1	1
5	1	2	2	0
6	4	0	0	-1

Examples of Fitness Functions

- ปัญหาการหาค่าที่เหมาะสมที่สุด (การหาค่าต่ำสุด/ค่าสูงสุด)
 - สำหรับการหาค่าสูงสุด (Maximization) ของฟังก์ชัน $f(x)$: $\text{fitness} = f(x)$
 - สำหรับการหาค่าต่ำสุด (Minimization) ของฟังก์ชัน $f(x)$: $\text{fitness} = \frac{1}{1+f(x)}$ (เพื่อให้ค่าที่น้อยกว่า ให้ผลลัพธ์เป็นค่าความเหมาะสมที่สูงกว่า)
- การแก้สมการ
 - สำหรับการแก้สมการ $ax^2 + bx + c = d$,โดยที่โครโมโซมแทนค่า (a, b, c, d):
 - $\text{Fitness} = \frac{1}{1+|ax^2+bx+c-d|}$
- การหาเส้นทางที่เหมาะสมที่สุด (เช่น ปัญหาการเดินทางของพนักงานขาย, TSP)
 - $\text{Fitness} = \frac{1}{\text{total distance}}$

Step 2: Evaluation (Fitness Function)

เป้าหมาย: วัดว่าโครโมโซมแต่ละชุด (คำตอบ) เข้าใกล้การแก้สมการได้มากน้อยเพียงใด:

วิธีการประเมิน:

- คำนวณค่าของแต่ละโครโมโซม: $Value = 2a + 4b + 3c - 2d$
- คำนวณค่าความคลาดเคลื่อน (Error): $Error = | 41 - Value |$
- แปลงค่าความคลาดเคลื่อนเป็นค่าความเหมาะสม (Fitness): ยิ่งค่าความคลาดเคลื่อนน้อยเท่าไร ค่าความเหมาะสมก็จะยิ่งดีขึ้นเท่านั้น.

$$Fitness = \frac{1}{1 + Error}$$

Step 2: Evaluation (Fitness Function)

$$2a + 4b + 3c - 2d = 41$$

<i>ID</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>Value</i>	<i>Error</i>	<i>Fitness</i>
1	4	3	2	0	26	15	0.0625
2	-3	5	0	-1	16	25	0.0385
3	2	3	-1	-2	17	24	0.0400
4	-3	5	1	1	15	26	0.0377
5	1	2	2	0	16	25	0.0385
6	4	0	0	-1	10	31	0.0313

- โครโมโซมที่ 1 มีค่าความเหมาะสม (fitness) ดีที่สุดในขณะนี้ (0.0625) แต่ยังคงห่างไกลจากเป้าหมาย
- คัดเลือกตัวที่เหมาะสมที่สุดเพื่อนำไปเป็นพ่อแม่พันธุ์ในขั้นตอนถัดไป

Step 3: Selection (Choosing Parents)

เป้าหมาย: เลือกโครโมโซม (คำตอบ) ที่จะมาเป็นพ่อแม่เพื่อสร้างประชากรรุ่นถัดไป.

- ต้องการให้คำตอบที่ดีกว่า (ค่าความเหมาะสมสูงกว่า) มีโอกาสถูกเลือกมากกว่า แต่ยังอนุญาตให้คำตอบที่ด้อยกว่าบางส่วนถูกเลือกด้วย เพื่อรักษาความหลากหลาย.

วิธีการที่นิยม: การเลือกแบบกงล้อรูเล็ต (Roulette Wheel Selection)

- โครโมโซมแต่ละตัวจะได้รับส่วนแบ่งบนกงล้อรูเล็ตตามสัดส่วนของค่าความเหมาะสม
- หมุนวงล้อ (หรือสุ่มตัวเลข) เพื่อเลือกโครโมโซมที่จะมาเป็นพ่อแม่ตัวถัดไป.



Step 3: Selection (Choosing Parents)

คำนวณความน่าจะเป็นในการเลือก

$$Total\ Fitness = 0.0625 + 0.0385 + 0.0400 + 0.0377 + 0.0385 + 0.0313 = 0.2485$$

<i>ID</i>	<i>Fitness</i>	<i>Probability (%)</i>
1	0.0625	25.1%
2	0.0385	15.5%
3	0.0400	16.1%
4	0.0377	15.2%
5	0.0385	15.5%
6	0.0313	12.6%

Step 3: Selection (Choosing Parents)

- คัดเลือกพ่อแม่ (Parents) 6 ราย (โดยการสุ่มตัวอย่างแบบกำหนดความน่าจะเป็น)
- **สมมติ**ว่าวงล้อรูเล็ตสุ่มเลือกได้ ดังนี้ :

- Parent 1: ID 1 $\longrightarrow$ [4, 3, 2, 0]
- Parent 2: ID 3 $\longrightarrow$ [2, 3, -1, -2]
- Parent 3: ID 5 $\longrightarrow$ [1, 2, 2, 0]
- Parent 4: ID 1 $\longrightarrow$ [4, 3, 2, 0] (again)
- Parent 5: ID 2 $\longrightarrow$ [-3, 5, 0, -1]
- Parent 6: ID 3 $\longrightarrow$ [2, 3, -1, -2] (again)

ได้พ่อแม่พันธุ์แม่พันธุ์ 6 ชุดสำหรับ
การผสมข้ามพันธุ์

Step 4: Crossover (Reproduction)

เป้าหมาย: ผสมโครโมโซมพ่อแม่สองสายเพื่อสร้างโครโมโซมลูกสายใหม่ โดยการสลับส่วนของยีน

Crossover Type	Best for	Keeps Order?	High Diversity?	Notes
Single-Point	General	Yes	Medium	Easy but may disrupt genes
Two-Point	General	Yes	Yes	More stable than single
Uniform	General	No	Yes	Very diverse
Arithmetic/Blend	Real-valued vars	No	Yes	Good for continuous spaces
Order Crossover	Permutations (TSP)	Yes	Yes	Keeps valid permutations

Step 4: Crossover (Reproduction)

Single-Point Crossover (การสืบพันธุ์แบบสลับสายตำแหน่งเดียว)

- เลือกจุดตัดหนึ่งจุด: กำหนดตำแหน่งที่จะตัดแบ่งบนสายโครโมโซมเพียงจุดเดียว
- สลับยีนหลังจุดตัด: ทำการแลกเปลี่ยนข้อมูลทางพันธุกรรม (ยีน) ระหว่างโครโมโซมพ่อและแม่ตั้งแต่ตำแหน่งหลังจุดตัดนั้นเป็นต้นไป
- ง่ายและรวดเร็ว: มีขั้นตอนการคำนวณที่ไม่ซับซ้อนและประมวลผลได้ไว
- อาจสูญเสียชุดข้อมูลสำคัญ: หากยีนที่ทำงานร่วมกันถูกแยกออกจากกันด้วยจุดตัด อาจทำให้ลักษณะเด่นที่ดีหายไป

ตัวอย่าง:

Parent A = [1, 2, | 3, 4] → Child = [1, 2, 7, 8]
Parent B = [5, 6, | 7, 8]

Step 4: Crossover (Reproduction)

Two-Point Crossover (การผสมข้ามสายพันธุ์แบบสองจุด)

- เลือกจุดตัดสองจุด
- สลับส่วนกลางระหว่างโครโมโซมพ่อและแม่
- ตัวอย่าง:

Parent A = [1, | 2, 3 |, 4]
Parent B = [5, | 6, 7 |, 8] → Child = [1, 6, 7, 4]

- รักษาโครงสร้างได้ดีกว่าการผสมแบบจุดเดียว
- ยังคงมีความเสี่ยงที่จะทำลายรูปแบบที่ดี (Patterns) หากเลือกจุดตัดไม่เหมาะสม

Step 4: Crossover (Reproduction)

Uniform Crossover (การผสมข้ามสายพันธุ์แบบสม่ำเสมอ)

- ยีนแต่ละตำแหน่งมีโอกาส 50% ที่จะถูกเลือกมาจากพ่อหรือแม่ฝ่ายใดฝ่ายหนึ่ง
- เปรียบเสมือนการโยนเหรียญเสี่ยงทายในทุกๆ ตำแหน่งของยีน

ตัวอย่าง :

Parent A = [1, 2, 3, 4]
Parent B = [5, 6, 7, 8] → Child might be [1, 6, 3, 8]

- มีการผสมผสานกันในระดับสูง; ช่วยส่งเสริมความหลากหลาย
- สามารถทำลายกลุ่มที่ผสมผสานกันอย่างลงตัวและมีประโยชน์ได้

Step 4: Crossover (Reproduction)

- วิธีการ: **การสลับสายจุดเดียว (Single-point crossover)**
 - สุ่มเลือกจุดสลับสาย (ตัวอย่างเช่น ระหว่างยีนตำแหน่งที่ 1 และ 2) และทำการสลับยีนระหว่างพ่อแม่ตั้งแต่จุดนั้นเป็นต้นไป
 - ตัวอย่างจุดสลับสาย: หลังยีนตัวที่ 2 (b) นั้นหมายความว่า: ลูกจะได้รับยีน 2 ตัวแรกจากพ่อแม่ A และได้รับยีน 2 ตัวสุดท้ายจากพ่อแม่ B

Pair	Parent A	Parent B	Crossover point	Child 1	Child 2
1	[4, 3, 2, 0]	[2, 3, -1, -2]	after gene 2	[4, 3, -1, -2]	[2, 3, 2, 0]
2	[1, 2, 2, 0]	[4, 3, 2, 0]	after gene 2	[1, 2, 2, 0]	[4, 3, 2, 0]
3	[-3, 5, 0, -1]	[2, 3, -1, -2]	after gene 2	[-3, 5, -1, -2]	[2, 3, 0, -1]

Step 4: Crossover (Reproduction)

รุ่นลูกหลังการครอสโอเวอร์ :

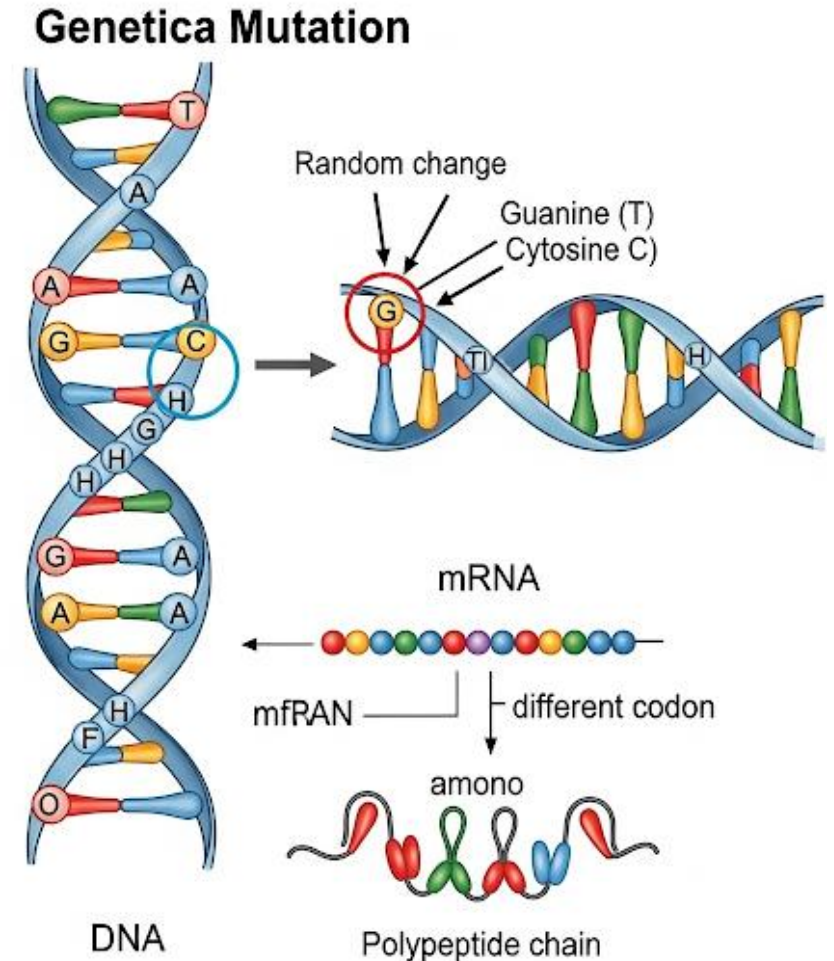
<i>Child ID</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>
1	4	3	-1	-2
2	2	3	2	0
3	1	2	2	0
4	4	3	2	0
5	-3	5	-1	-2
6	2	3	0	-1

ใช้การสุ่มกลายพันธุ์ (Random Mutations) เพื่อรักษาความหลากหลายและแสวงหาคำตอบที่หลากหลายยิ่งขึ้น

Step 5: Mutation (Genetic Diversity Injection)

เป้าหมาย:

- การกลายพันธุ์คือการสุ่มเปลี่ยนแปลงเล็กน้อยในยีนของรุ่นลูก
- ช่วยรักษาความหลากหลายของประชากรและช่วยป้องกันไม่ให้เกิดการติดหล่มอยู่ที่ค่าคำตอบที่ดีที่สุดเฉพาะจุด (local optima)



Step 5: Mutation (Genetic Diversity Injection)

วิธีการทำงาน:

1. อัตราการกลายพันธุ์ (Mutation Rate):

- โดยปกติจะมีค่าน้อย เช่น 5% ถึง 10%
- หมายความว่ายีนแต่ละตัวมีโอกาสเล็กน้อยที่จะเกิดการเปลี่ยนแปลง

2. กลไกการกลายพันธุ์ (Mutation Mechanism):

- สุ่มเลือกยีนมาหนึ่งตัว
- เปลี่ยนค่าของยีนนั้น โดยอาจจะเป็นการสุ่มค่าใหม่ทั้งหมด หรือปรับเปลี่ยนเพียงเล็กน้อย

Step 5: Mutation (Genetic Diversity Injection)

ทำการกลายพันธุ์ (Mutation) กับลูกทั้ง 6 คน

- ตัวอย่างเช่น: อัตราการกลายพันธุ์ = 20% ($0.2 * 6 = 1.2 \rightarrow 2$)
- ทำการสุ่มกลายพันธุ์ลูก 2 คน จากทั้งหมด 6 คน.

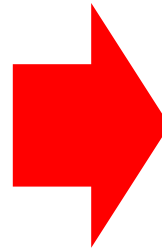
Child ID	Before Mutation	Mutation Applied	After Mutation
1	[4, 3, -1, -2]	Change gene c from -1 $\rightarrow$ 1	[4, 3, 1, -2]
5	[-3, 5, -1, -2]	Change gene a from -3 $\rightarrow$ -1	[-1, 5, -1, -2]

รุ่นลูกคนอื่นๆ ยังคงไม่เปลี่ยนแปลง

Step 5: Mutation (Genetic Diversity Injection)

- ประชากรชุดใหม่หลังการกลายพันธุ์

<i>Child ID</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>
1	4	3	-1	-2
2	2	3	2	0
3	1	2	2	0
4	4	3	2	0
5	-3	5	-1	-2
6	2	3	0	-1



<i>Child ID</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>
1	4	3	1	-2
2	2	3	2	0
3	1	2	2	0
4	4	3	2	0
5	-1	5	-1	-2
6	2	3	0	-1

เมื่อการกลายพันธุ์เสร็จสิ้น กลับไปที่: ขั้นตอนที่ 2: การประเมินผล (ความเหมาะสม)

Generation 2

ประเมินรุ่นที่ 2

Child ID	Chromosome (a, b, c, d)	Value	Error	Fitness
1	[4, 3, 1, -2]	$8 + 12 + 3 + 4 = \mathbf{27}$	14	0.0667
2	[2, 3, 2, 0]	$4 + 12 + 6 - 0 = \mathbf{22}$	19	0.0500
3	[1, 2, 2, 0]	$2 + 8 + 6 - 0 = \mathbf{16}$	25	0.0385
4	[4, 3, 2, 0]	$8 + 12 + 6 - 0 = \mathbf{26}$	15	0.0625
5	[-1, 5, -1, -2]	$-2 + 20 - 3 + 4 = \mathbf{19}$	22	0.0435
6	[2, 3, 0, -1]	$4 + 12 + 0 + 2 = \mathbf{18}$	23	0.0417

- ความเหมาะสม (Fitness) ที่ดีที่สุดในรุ่นที่ 2: 0.0667 (ลูกคนที่ 1)
- ยังห่างไกลจากความสมบูรณ์แบบ แต่มีการพัฒนาขึ้นเล็กน้อย
- ทำขั้นตอนซ้ำ: การคัดเลือก $\rightarrow$ การผสมข้ามสายพันธุ์ $\rightarrow$ การกลายพันธุ์ อีกครั้งสำหรับรุ่นถัดไป

Sample result

พัฒนาโค้ด Python เพื่อทำซ้ำสำหรับจำนวนรุ่น (generation) ที่มากขึ้น เราจะได้ผลลัพธ์ของ GA (Genetic Algorithm) ดังนี้

=== First Generation Population ===

Chromosome 1: [-6, -6, -3, -4] | Value = -37 | Error = 78 | Fitness = 0.0127

Chromosome 2: [-9, -1, -2, -4] | Value = -20 | Error = 61 | Fitness = 0.0161

Chromosome 3: [5, 3, -8, -3] | Value = 4 | Error = 37 | Fitness = 0.0263

Chromosome 4: [1, 5, -6, 2] | Value = 0 | Error = 41 | Fitness = 0.0238

Chromosome 5: [7, -7, 6, 5] | Value = -6 | Error = 47 | Fitness = 0.0208

Chromosome 6: [1, 10, 10, 4] | Value = 64 | Error = 23 | Fitness = 0.0417

=== Last Generation Population ===

Chromosome 1: [1, 5, 9, 4] | Value = 41 | Error = 0 | Fitness = 1.0

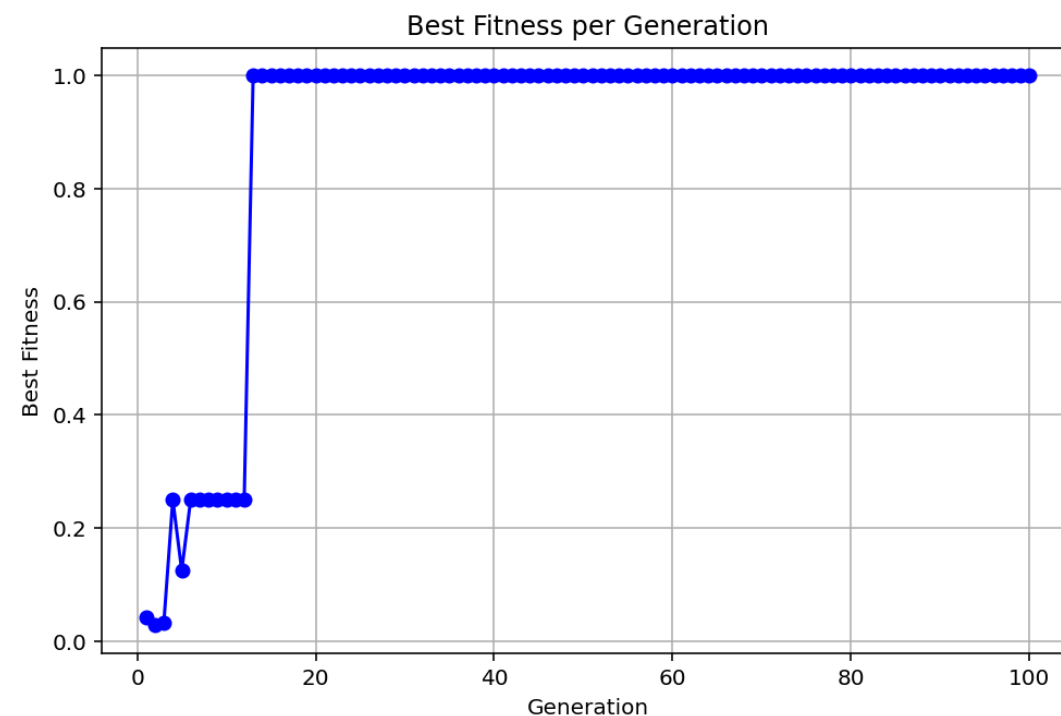
Chromosome 2: [1, 5, 9, 4] | Value = 41 | Error = 0 | Fitness = 1.0

Chromosome 3: [1, 5, 9, 4] | Value = 41 | Error = 0 | Fitness = 1.0

Chromosome 4: [1, 5, 9, 4] | Value = 41 | Error = 0 | Fitness = 1.0

Chromosome 5: [1, 5, 9, -3] | Value = 55 | Error = 14 | Fitness = 0.0667

Chromosome 6: [1, 5, 9, 4] | Value = 41 | Error = 0 | Fitness = 1.0



Sample result

พัฒนาโค้ด Python เพื่อทำซ้ำสำหรับจำนวนรุ่น (generation) ที่มากขึ้น เราจะได้ผลลัพธ์ของ GA (Genetic Algorithm) ดังนี้

=== First Generation Population ===

Chromosome 1: [10, -8, -8, -10] | Value = -16 | Error = 57 | Fitness = 0.0172

Chromosome 2: [5, -4, -1, -1] | Value = -7 | Error = 48 | Fitness = 0.0204

Chromosome 3: [-6, -4, 8, -3] | Value = 2 | Error = 39 | Fitness = 0.025

Chromosome 4: [-9, -9, -3, -3] | Value = -57 | Error = 98 | Fitness = 0.0101

Chromosome 5: [-7, 10, -5, -9] | Value = 29 | Error = 12 | Fitness = 0.0769

Chromosome 6: [-8, -4, 9, 9] | Value = -23 | Error = 64 | Fitness = 0.0154

=== Last Generation Population ===

Chromosome 1: [8, -4, 7, -10] | Value = 41 | Error = 0 | Fitness = 1.0

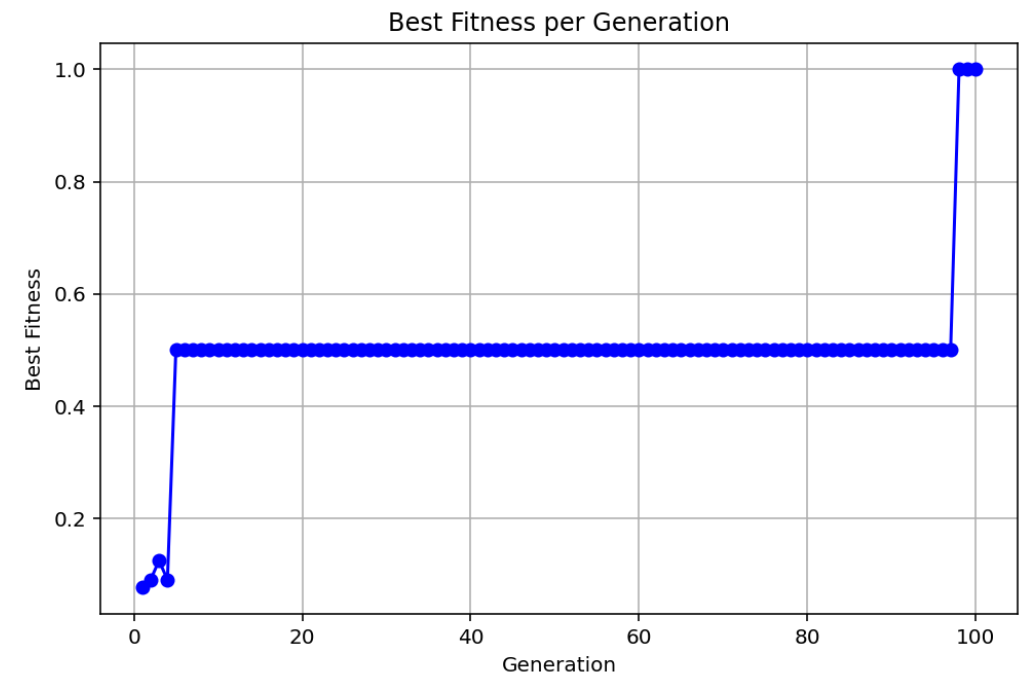
Chromosome 2: [8, -4, 7, -10] | Value = 41 | Error = 0 | Fitness = 1.0

Chromosome 3: [8, -4, 7, -10] | Value = 41 | Error = 0 | Fitness = 1.0

Chromosome 4: [8, -4, 7, -10] | Value = 41 | Error = 0 | Fitness = 1.0

Chromosome 5: [8, -4, 7, -10] | Value = 41 | Error = 0 | Fitness = 1.0

Chromosome 6: [8, -4, 7, -10] | Value = 41 | Error = 0 | Fitness = 1.0



Sample result

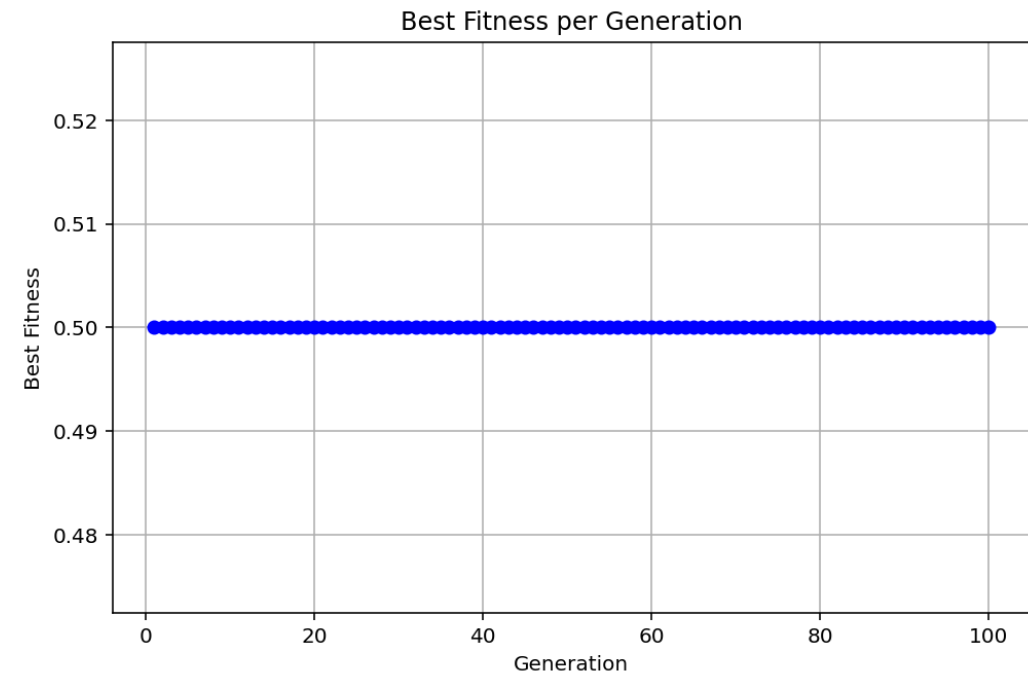
พัฒนาโค้ด Python เพื่อทำซ้ำสำหรับจำนวนรุ่น (generation) ที่มากขึ้น เราจะได้ผลลัพธ์ของ GA (Genetic Algorithm) ดังนี้

=== First Generation Population ===

Chromosome 1: [-1, 2, 1, -7] | Value = 23 | Error = 18 | Fitness = 0.0526
Chromosome 2: [-3, -3, 10, -5] | Value = 22 | Error = 19 | Fitness = 0.05
Chromosome 3: [-3, 2, -10, -2] | Value = -24 | Error = 65 | Fitness = 0.0152
Chromosome 4: [-3, 6, -6, -4] | Value = 8 | Error = 33 | Fitness = 0.0294
Chromosome 5: [-6, -7, -7, -1] | Value = -59 | Error = 100 | Fitness = 0.0099
Chromosome 6: [4, 8, -2, -3] | Value = 40 | Error = 1 | Fitness = 0.5

=== Last Generation Population ===

Chromosome 1: [4, 8, -2, -3] | Value = 40 | Error = 1 | Fitness = 0.5
Chromosome 2: [4, 8, -2, -3] | Value = 40 | Error = 1 | Fitness = 0.5
Chromosome 3: [4, 8, -2, -3] | Value = 40 | Error = 1 | Fitness = 0.5
Chromosome 4: [-4, 8, -2, -3] | Value = 24 | Error = 17 | Fitness = 0.0556
Chromosome 5: [4, 8, -2, -3] | Value = 40 | Error = 1 | Fitness = 0.5
Chromosome 6: [4, 8, -2, -3] | Value = 40 | Error = 1 | Fitness = 0.5



Typical Applications

- การออกแบบทางวิศวกรรม (Engineering design)
- การเรียนรู้ของเครื่อง (Machine learning) (การคัดเลือกคุณลักษณะ, การปรับปรุงโครงข่ายประสาทเทียมให้มีประสิทธิภาพสูงสุด)
- ปัญหาการจัดตารางเวลาและการวางแผน (Scheduling and planning problems)
- ปัญญาประดิษฐ์ในเกม (Game AI)
- ระบบหุ่นยนต์และการควบคุม (Robotics and control systems)

Advantages

- เหมาะสำหรับปัญหาที่มีความซับซ้อน ไม่เป็นเส้นตรง และมีหลายรูปแบบ (Multi-modal)
- ใช้งานได้ดีเมื่อไม่มีค่าอนุพันธ์หรือความชัน (Gradient) ให้ใช้
- มีความยืดหยุ่นสำหรับปัญหาการหาค่าที่เหมาะสมที่สุด (Optimization) ในรูปแบบต่าง ๆ
- สามารถหลีกเลี่ยงการติดอยู่ในจุดต่ำสุดเฉพาะที่ (Local minima) ได้

Disadvantages

- อาจต้องใช้การประเมินหลายครั้ง (ล่าช้า)
- อาจไม่พบจุดที่เหมาะสมที่สุด (global optimum) ที่แท้จริง
- การออกแบบฟังก์ชันความเหมาะสม (Fitness function) มีความสำคัญมากและบางครั้งก็ท้าทาย
- ผลลัพธ์อาจเปลี่ยนแปลงไปตามการปรับจูนพารามิเตอร์

Python Library: PyGAD

ภาพรวม: ไลบรารีนี้ขึ้นชื่อเรื่องความใช้งานง่าย และเป็นแนวทางที่ตรงไปตรงมาในการนำอัลกอริทึมพันธุกรรม (Genetic Algorithms) ไปใช้งาน โดยไม่จำเป็นต้องมีความเข้าใจลึกซึ้งเกี่ยวกับกลไกภายใน เหมาะอย่างยิ่งสำหรับผู้เริ่มต้น.

- **Strengths:**

- เริ่มต้นใช้งานได้ง่ายมาก
- รองรับรูปแบบการผสมสายพันธุ์ (Crossover) และการกลายพันธุ์ (Mutation) หลากหลายรูปแบบที่มีมาให้ในตัว
- เหมาะสำหรับปัญหาการหาค่าที่เหมาะสมที่สุดแบบมาตรฐาน (Standard optimization problems) และการประยุกต์ใช้กับแมชชีนเลิร์นนิง (เช่น การปรับจูนไฮเปอร์พารามิเตอร์).

Python Library : DEAP

DEAP (Distributed Evolutionary Algorithms in Python)

- **ภาพรวม:** เฟรมเวิร์กที่มีความยืดหยุ่น เหมาะสำหรับการวิจัยและการคำนวณเชิงวิวัฒนาการ (Evolutionary Computations) ที่มีความซับซ้อน เนื่องจากมีการออกแบบโครงสร้างในรูปแบบโมดูล (Modular design)
- **Strengths:**
 - สามารถปรับแต่งได้ในระดับสูง
 - มีประสิทธิภาพและการขยายตัว (Scalability) ที่ดี โดยเฉพาะความสามารถในการทำงานแบบขนาน (Parallelization)
 - มีการจัดทำเอกสารคู่มือไว้เป็นอย่างดีและมีชุมชนผู้ใช้งานจำนวนมาก.

Python Library: TPOT

TPOT (Tree-based Pipeline Optimization Tool)

- **ภาพรวม:** มีความเฉพาะทางสูง แต่ TPOT ใช้การเขียนโปรแกรมเชิงพันธุกรรม (Genetic Programming) เพื่อสร้างและปรับแต่งท่อส่งข้อมูลการเรียนรู้ของเครื่อง (Machine Learning Pipelines) โดยอัตโนมัติ TPOT คือตัวเลือกที่ดีหากเป้าหมายคือการทำ Automated Machine Learning (AutoML).
- **Strengths:**
 - เชี่ยวชาญด้านการสร้างท่อส่งข้อมูลการเรียนรู้ของเครื่องแบบอัตโนมัติ
 - สามารถค้นพบท่อส่งข้อมูลที่มีความซับซ้อนและมีประสิทธิภาพสูงได้.

Python Library: Scikit-learn

Scikit-learn (with some custom implementation)

- **ภาพรวม:** ไม่มีโมดูลสำหรับขั้นตอนวิธีพันธุกรรม (Genetic Algorithm - GA) โดยเฉพาะ แต่สามารถนำองค์ประกอบของ GA มาประยุกต์ใช้หรือเชื่อมต่อกันได้ โดยเฉพาะอย่างยิ่งในการหาค่าไฮเปอร์พารามิเตอร์ที่เหมาะสมที่สุด (Hyperparameter Optimization)
- **Strengths:**
 - มีระบบนิเวศของเครื่องมือการเรียนรู้ของเครื่อง (Machine Learning) ที่กว้างขวาง
 - มีอัลกอริทึมที่เสถียรและผ่านการทดสอบมาอย่างดี สำหรับส่วนอื่น ๆ ในขั้นตอนการประมวลผลข้อมูล (ML Pipeline).